

4 — Getter: a read that secretly runs a function

OBJECTS ACCESSOR INTERCEPTION

Q. `article.byline` has no parentheses, so why does it return a freshly built string every time you read it?

ANSWER. Because `byline` is not a stored value. It is a getter: a property defined with the `get` keyword that secretly runs a function on every read. To the call site it looks identical to plain data, but inside, code is executing.

WHY IT WORKS. Reading `article.author` is a plain data-property access — JavaScript hands back the stored string and nothing runs. Reading `article.byline` looks the same, but the `get` keyword makes it an accessor: reading it runs `get byline()` and returns whatever the function produces. Because it recomputes from the current `author` on every access, it can never go stale. This is the first crack in the passive read.

```
const article = {  
  author: "Ada Lovelace",  
  data property  
  get byline() {  
    runs on every read  
    return "By " + this.author;  
    fresh value  
  }  
};  
console.log(article.byline);  
Lovelace" (no parentheses)  
article.author = "Grace Hopper";  
underlying data  
console.log(article.byline);  
Hopper" (recomputed)
```

examples/byline.js

TWO READS, TWO OUTCOMES

`article.author`



stored string

DATA PROPERTY · NO CODE RUNS

RUNS CODE

article.byline → **runs get byline()**

ACCESSOR · RECOMPUTES EVERY READ

- ① **Summary.** A getter is a function disguised as a property: reading it runs code and returns a fresh value, so it can never go stale. The catch is that a getter guards one property you named in advance — what catches a property you didn't name?